Java

1D Arrays , 2D arrays, string literals

```
import java.util.Scanner;

public class Array {
    public static void main(String[] args) {

        // System.out.println("Array");

        // array => data structure , non primitive , continous memory location
, using index
        // 1000 records => index value store arr[999];

        // single dimensional
        // int arr[] = {1,2,3,4,5};
        // System.out.println(arr.length);
        // arr[0] = 10;

        // System.out.println(arr[0]);
        // System.out.println(arr[1]);
        // System.out.println(arr[2]);
        // System.out.println(arr[3]);
        // System.out.println(arr[4]);
        // i =0 ,i<5, i++
        // for(int i=0; i<arr.length; i++){
        //     arr[i] = i+10;
        //     System.out.println(arr[i]);
        // }

        // int data;

        Scanner inp = new Scanner(System.in);
        // System.out.print("Enter array size: ");
        // data = inp.nextInt();


        // int userArray[] = new int[data];

        // for(int i=0; i<data; i++){
        //     System.out.print("Enter " + i + "th element");
        //     userArray[i] = inp.nextInt();
        // }

        // for(int i=0; i<userArray.length; i++){
        //     System.out.print(userArray[i]+" ");
        // }
```

```
// two dimensional array => row and column

// int [][] arr = {{1,2}, {3,4},{5,6}};
// int row , col;
// row = inp.nextInt();
// col = inp.nextInt();

// int [][] userArray = new int[row][col];

// for(int i=0; i<row; i++){

//     for(int j=0; j<col; j++){
//         System.out.print("Enter "+ i+" row and "+j+" column: ");
//         userArray[i][j] = inp.nextInt();
//     }
// }


// arr[0][0] = 18;
// System.out.println(arr[0][0]);
// System.out.println(arr[0][1]);
// System.out.println(arr[1][0]);
// arr[1][1]=7;
// System.out.println(arr[1][1]);

// System.out.println(arr.length);
// System.out.println(arr[0].length);

// for(int i=0; i<userArray.length; i++){

//     for(int j=0; j<userArray[0].length; j++){
//         System.out.print(userArray[i][j] +" ");
//     }
//     System.out.println();

// }


//jagged array => row fixed but column different

int [][] jagged = new int[3][];

jagged[0] = new int[2];
jagged[1] = new int[4];
jagged[2] = new int[3];

// for(int i=0; i<jagged.length; i++){
//     for(int j=0; j<jagged[i].length; j++){
```

```
        //          System.out.print("Enter "+i+" row and "+j+" column: ");
        //          jagged[i][j] = inp.nextInt();
        //      }
        // }
         for(int i=0; i<jagged.length; i++){

            for(int j=0; j<jagged[i].length; j++){
                System.out.print(jagged[i][j]+" ");

            }
            System.out.println();
        }


        int arr[] = {1,2,3,10,40,60,10,20};

        int max = arr[0];
        for(int i=0; i<arr.length; i++){

            if(arr[i] > max){
                max = arr[i];
            }
        }

        System.out.println(max);

        // string => non- primitive , java=> class

        // string literal => stores in string constant pool
        String name ="Priyanka";
        String userName ="Priyanka";

        System.out.println(name.equals(userName));
        System.out.println(name == userName);

        String difName = new String("Priyanka");


        System.out.println(name.equals(difName)); // check only value
        System.out.println(name == difName); //check for reference

        // two dimensional
        // jagged array

    }

}
```

📘 Theory Notes

1. Arrays

- **Definition**: A data structure that stores elements of the same type in contiguous memory locations.
- **Single-dimensional array**:

```
int arr[] = {1,2,3,4,5};
```

Access with arr[index]. Index starts at 0.

- **Two-dimensional array**:

```
int[][] arr = {{1,2}, {3,4}};
```

Access with arr[row][col].

- **Jagged array**: Array of arrays with different column sizes.

```
int[][] jagged = new int[3][];
jagged[0] = new int[2];
jagged[1] = new int[4];
jagged[2] = new int[3];
```

2. Loops with Arrays

- **for loop** → iterate through array elements.
- **nested for loop** → used for 2D arrays.
- Example:

```
for(int i=0; i<arr.length; i++) {
   System.out.println(arr[i]);
}
```

3. Finding Maximum in Array

- Initialize max = arr[0].
- Traverse array, update max if a larger value is found.
- Example:

```
int max = arr[0];
for(int i=0; i<arr.length; i++) {
   if(arr[i] > max) max = arr[i];
}
```

4. Strings in Java

- **String literal** → stored in String Constant Pool.

String name = "Priyanka";

- **String object** → created using new, stored in heap.

String difName = new String("Priyanka");

- **equals()** → compares values.
- **==** → compares references (memory addresses).

5. Key Differences

- equals() → true if values match.
- == → true only if both references point to the same object.

Practice Tasks

Short Answer

1. What is the difference between a 2D array and a jagged array?
2. Why does == return false when comparing string objects created with new?
3. How do you find the length of an array in Java?
4. What is stored in the String Constant Pool?

Coding Exercises

1. Write a program to input 5 integers into an array and print their sum.
2. Create a 2D array of size 3x3 and fill it with user input, then print it in matrix form.
3. Implement a jagged array where the first row has 2 columns, second row has 3, and third row has 4.
4. Write a program to find the smallest element in an array.
5. Compare two strings using both equals() and == and explain the difference.